

≡ ChatGPT ▾ ✦ Get Plus ✕

This is a copy of a conversation between ChatGPT & Anonymous.
[Report conversation](#)

Generate Test Strategy Document

- take 5 questions step by step once after answering proceed with next question
- Test strategy document should be enterprise following industry standard

Great — we'll build this as a proper **enterprise-level Test Strategy Document**, step by step, just like in real projects.

I'll ask **5 structured questions one-by-one**, and based on your answers, I'll progressively generate the document.

✅ Step 1 – Project Overview

To start the Test Strategy, I need core project details.

Question 1: Project Information

Please provide:

1. **Project/Application Name**
2. **Domain** (e.g., Banking, Healthcare, E-commerce, etc.)
3. **Application Type** (Web / Mobile / API / Desktop)
4. **Brief Description of the Application**
5. **Key Modules / Features** (e.g., Login, Payments, Reports, etc.)

Reply with these details, and I'll generate the **Project Overview section** and move to the next question.

1. LMS
2. Education
3. Web Application
4. Student Learning Management System
5. Login, Enrollment, Course, Exams

Perfect — based on your input, here is the **enterprise-standard Test Strategy – Section 1.**

Test Strategy Document – LMS (Learning Management System)

1. Project Overview

1.1 Project Name

Learning Management System (LMS)

1.2 Domain

Education

1.3 Application Type

Web Application

1.4 Application Description

The Learning Management System (LMS) is a web-based platform designed to facilitate online education and training. It enables students to access courses, enroll in programs, attempt examinations, and track their learning progress. The system also supports administrative functionalities for managing courses, users, and assessments.

1.5 Key Modules in Scope

- **User Authentication (Login)**
 - **Course Enrollment**
 - **Course Management**
 - **Examination Management**
-

Step 2 – Scope of Testing

Question 2: Scope Definition

Please provide:

1. In-Scope Testing Types (choose or add):

- **Functional Testing**
- **Integration Testing**
- **System Testing**

- Regression Testing
 - Smoke/Sanity Testing
 - API Testing
 - UI Testing
 - Accessibility Testing
 - Performance Testing
 - Security Testing
- 2. Out-of-Scope Items** (if any):
(e.g., third-party integrations, payment gateway, etc.)
- 3. Test Environment Availability**
(e.g., Dev, QA, UAT)
-

Reply with these details, and I'll generate the **Scope of Testing** section and proceed to Step 3.

In-Scope Testing Types

Functional Testing
Integration Testing
System Testing
Regression Testing
Smoke/Sanity Testing
API Testing
UI Testing
Accessibility Testing

2. Out-of-Scope Items
third-party integrations,
payment gateway
3. Test Environment Availability
Dev, QA, UAT

Great — this is shaping up like a solid enterprise Test Strategy.

Here's **Section 2: Scope of Testing** based on your inputs.

2. Scope of Testing

2.1 In-Scope Testing

The following testing types are included within the scope of the LMS Test Strategy:

- **Functional Testing**
Validation of all core functionalities including Login, Enrollment, Course Management, and Examination modules against business requirements.
 - **Integration Testing**
Verification of interactions between modules such as Enrollment with Course Management and Exams with User Data.
 - **System Testing**
End-to-end validation of the complete LMS application in an environment that closely mimics production.
 - **Regression Testing**
Execution of regression test suites to ensure that new changes or enhancements do not impact existing functionalities.
 - **Smoke/Sanity Testing**
High-level validation of critical functionalities to ensure build stability before detailed testing.
 - **API Testing**
Validation of backend services to ensure correct data exchange, response validation, and error handling.
 - **UI Testing**
Verification of user interface elements for consistency, usability, responsiveness, and adherence to design standards.
 - **Accessibility Testing**
Ensuring the application meets accessibility standards (such as WCAG guidelines) for users with disabilities.
-

2.2 Out-of-Scope Testing

The following items are excluded from the current testing scope:

- Third-party integrations
 - Payment gateway functionalities
-

2.3 Test Environments

Testing activities will be conducted across the following environments:

- **Development (Dev)** – Initial testing and developer validation

- **Quality Assurance (QA)** – Formal testing cycles including functional, integration, and regression testing
 - **User Acceptance Testing (UAT)** – Business validation and end-user acceptance
-

✓ Step 3 – Test Approach

Now we define *how* testing will be executed.

Question 3: Test Approach Details

Please provide:

1. Testing Methodology

(Agile / Waterfall / Hybrid)

2. Automation Scope

- % of automation (approx)
- What will be automated? (e.g., regression, smoke, API)

3. Tools Used

(Example: Selenium, TestNG, Cucumber, JIRA, Postman, etc.)

4. Test Data Strategy

(e.g., Real data / Masked data / Synthetic data)

Reply with these details, and I'll generate a **strong enterprise-level Test Approach section** next.

Testing Methodology - Waterfall

Automation Scope - % of automation

What will be automated? regression, smoke, API

Tools Used

Selenium, TestNG, Cucumber, JIRA, Postman

Test Data Strategy

Real data ,Masked data, Synthetic data

Excellent — this is aligning well with an enterprise-grade Test Strategy.

Here's **Section 3: Test Approach** crafted to industry standards.

3. Test Approach

3.1 Testing Methodology

The project will follow the **Waterfall Model**, where testing activities are executed in a sequential manner aligned with defined phases such as Requirements, Design, Development, Testing, and Deployment.

- Test planning and design will begin after requirement finalization.
 - Test execution will commence after the development phase is completed.
 - Each phase will have defined entry and exit criteria to ensure quality gates are met before progressing.
-

3.2 Testing Levels

The following levels of testing will be performed:

- **Unit Testing** – Conducted by developers prior to QA handover
 - **Integration Testing** – Validation of module interactions
 - **System Testing** – Complete end-to-end validation of LMS workflows
 - **User Acceptance Testing (UAT)** – Performed by business stakeholders
-

3.3 Automation Strategy

Automation will be implemented to improve efficiency, repeatability, and coverage of critical test scenarios.

- **Automation Coverage:** Approximately **60–70%** of the regression suite will be automated
 - **Automation Scope Includes:**
 - Smoke test suite for build validation
 - Regression test suite for core LMS functionalities
 - API test automation for backend validation
 - **Automation Execution:**
 - Automated tests will be integrated into CI/CD pipelines (where applicable)
 - Scheduled and on-demand execution will be supported
-

3.4 Tools and Technologies

The following tools will be used across testing activities:

- **Automation Testing:** Selenium WebDriver

- **Framework:** TestNG, Cucumber (BDD approach)
 - **Test Management & Defect Tracking:** JIRA
 - **API Testing:** Postman
-

3.5 Test Data Management

A robust test data strategy will be followed to ensure comprehensive coverage and data security:

- **Real Data**
Used in controlled environments where necessary for realistic validation
 - **Masked Data**
Sensitive data will be anonymized to comply with data privacy regulations
 - **Synthetic Data**
Artificially generated data will be used for edge cases, negative scenarios, and scalability
 - Test data will be version-controlled and refreshed periodically to maintain consistency across test cycles
-

Step 4 – Test Environment & Infrastructure

Now we define the technical setup.

Question 4: Environment & Infra Details

Please provide:

1. **Browsers to Support**
(e.g., Chrome, Firefox, Edge, Safari)
 2. **Operating Systems**
(e.g., Windows, Mac, Linux)
 3. **Build Deployment Process**
(e.g., manual / CI-CD / Jenkins)
 4. **Any Performance or Security Tools (if planned)**
(optional — say NA if not applicable)
-

Reply with these details, and I'll generate the next section.

Browsers to Support - Chrome, Firefox, Edge, Safari

Operating Systems - Windows, Mac

Build Deployment Process CI-CD

Any Performance or Security Tools - NA

Great — we're building a strong, enterprise-grade document step by step.

Here's **Section 4: Test Environment & Infrastructure**.

4. Test Environment & Infrastructure

4.1 Supported Browsers

The LMS application will be validated across multiple browsers to ensure cross-browser compatibility and consistent user experience:

- Google Chrome
 - Mozilla Firefox
 - Microsoft Edge
 - Apple Safari
-

4.2 Supported Operating Systems

Testing will be conducted on the following operating systems:

- Windows
- macOS

This ensures compatibility across major user platforms.

4.3 Test Environment Setup

The application will be deployed and tested across the following environments:

- **Development (Dev)** – Initial builds for developer validation
- **Quality Assurance (QA)** – Dedicated environment for formal testing cycles
- **User Acceptance Testing (UAT)** – Environment for business validation

Each environment will have controlled configurations to ensure stability and consistency during testing.

4.4 Build & Deployment Process

- The project follows a **CI/CD (Continuous Integration and Continuous Deployment)** pipeline.
- Code changes are integrated frequently and deployed automatically to relevant environments.
- Automated smoke tests will be triggered post-deployment to validate build stability.
- Rollback mechanisms will be in place in case of critical failures.

4.5 Infrastructure & Tools

- Test environments will be maintained with necessary application servers, databases, and network configurations.
- Testers will have access to required tools such as Selenium, TestNG, Cucumber, JIRA, and Postman.
- Version control and build pipelines will ensure traceability and consistency.

4.6 Performance & Security Testing Tools

- Currently, **Performance and Security Testing tools are not in scope** for this phase.
- These may be incorporated in future phases based on project requirements.

Step 5 – Test Management, Risks & Deliverables

Final step — this is where we make it **enterprise complete**.

Question 5: Final Details

Please provide (or I can suggest defaults if you prefer):

- 1. Entry Criteria for Testing**
(When should testing start?)
- 2. Exit Criteria**
(When can testing be considered complete?)
- 3. Key Risks**
(e.g., environment issues, data issues, timeline constraints)
- 4. Deliverables**
(e.g., Test Plan, Test Cases, Test Reports, Defect Reports)

Reply with these, and I'll generate the **final section + complete polished Test Strategy Document** ready to use.

Suggest this

Perfect — I'll define **enterprise-standard, industry-aligned defaults** for you and complete the final section.

5. Test Management, Risks & Deliverables

5.1 Entry Criteria

Testing activities will commence once the following conditions are met:

- Requirements are reviewed, approved, and baselined
 - Test Strategy and Test Plan are approved by stakeholders
 - Test environment (QA) is set up and stable
 - Application build is deployed successfully in QA
 - Test data is prepared and available
 - Test cases are designed, reviewed, and approved
-

5.2 Exit Criteria

Testing will be considered complete when:

- All planned test cases are executed
 - **95–100%** of test cases are passed
 - All **Critical (S1)** and **High (S2)** defects are resolved and verified
 - No open defects with severity S1/S2
 - Regression testing is completed successfully
 - Test summary report is prepared and approved
 - Stakeholder sign-off is obtained
-

5.3 Risk Management

The following risks have been identified along with mitigation strategies:

Risk	Impact	Mitigation Strategy
Unstable test environment	Delays in execution	Maintain dedicated QA environment and perform smoke checks
Incomplete/unclear requirements	Incorrect test coverage	Conduct requirement reviews and clarification sessions
Test data unavailability	Execution blockage	Prepare synthetic and masked data in advance
Tight project timelines	Reduced test coverage	Prioritize P1/P2 test cases and automate regression
Integration defects	System instability	Perform early integration testing

5.4 Test Deliverables

The following deliverables will be produced during the testing lifecycle:

- Test Strategy Document
- Test Plan
- Test Scenarios & Test Cases
- Test Data Documents
- Automation Scripts (Selenium, API)
- Test Execution Reports
- Defect Reports (via JIRA)
- Test Summary Report
- UAT Sign-off Document

 Final Output – Completed Test Strategy Document

You now have a **complete enterprise-level Test Strategy Document** with:

1. Project Overview
2. Scope of Testing
3. Test Approach
4. Test Environment & Infrastructure
5. Test Management, Risks & Deliverables

 **If you want next level (recommended for interviews / real projects):**

I can also help you with:

- Full Test Plan Document
- RTM (Requirement Traceability Matrix)
- Detailed Test Cases (Excel format)
- Automation Framework Structure (Selenium + Cucumber + TestNG)
- Sample CI/CD integration (Jenkins pipeline)

Just tell me 👍

Full Test Plan Document

Here is a **complete enterprise-level Test Plan Document** for your LMS project, aligned with industry standards (IEEE 829 / ISO style).

TEST PLAN DOCUMENT – LMS (Learning Management System)

1. Introduction

1.1 Purpose

This Test Plan defines the strategy, scope, resources, schedule, and activities required to test the Learning Management System (LMS). It ensures the application meets functional and non-functional requirements with high quality.

1.2 Scope

This document covers testing of the LMS web application including core modules:

- Login
 - Enrollment
 - Course Management
 - Exams
-

2. Test Items

The following modules/features will be tested:

- User Authentication (Login)
 - Course Enrollment
 - Course Management
 - Examination Module
 - APIs supporting above modules
 - UI components and workflows
-

3. Features to be Tested

- User login/logout functionality
 - Course enrollment workflows
 - Course creation and access
 - Exam creation, attempt, and evaluation
 - Data validation and error handling
 - API request/response validation
 - UI responsiveness and compatibility
 - Accessibility compliance
-

4. Features Not to be Tested

- Third-party integrations
 - Payment gateway functionalities
 - Performance and security testing (out of current scope)
-

5. Test Approach

5.1 Testing Methodology

- Waterfall model
- Sequential execution after development completion

5.2 Testing Types

- Functional Testing
- Integration Testing
- System Testing
- Regression Testing

- Smoke/Sanity Testing
- API Testing
- UI Testing
- Accessibility Testing

5.3 Automation Strategy

- ~60–70% automation coverage
 - Automated suites:
 - Smoke tests
 - Regression tests
 - API tests
 - Tools:
 - Selenium WebDriver
 - TestNG
 - Cucumber (BDD)
 - Postman
-

6. Test Environment

6.1 Environments

- Dev
- QA
- UAT

6.2 Browsers

- Chrome
- Firefox
- Edge
- Safari

6.3 Operating Systems

- Windows
- macOS

6.4 Deployment

- CI/CD pipeline-based deployment
 - Automated smoke validation post deployment
-

7. Entry and Exit Criteria

7.1 Entry Criteria

- Requirements approved
- Test cases reviewed
- QA environment ready
- Build deployed successfully
- Test data available

7.2 Exit Criteria

- 95–100% test execution completed
 - No open Severity S1/S2 defects
 - Regression completed
 - Test summary report approved
 - Stakeholder sign-off received
-

8. Test Deliverables

- Test Plan
 - Test Strategy
 - Test Cases & Scenarios
 - Test Data
 - Automation Scripts
 - Defect Reports
 - Test Execution Reports
 - Test Summary Report
 - UAT Sign-off
-

9. Test Schedule (Sample)

Phase	Activity	Duration
Phase 1	Test Planning	2 Days
Phase 2	Test Design	4 Days
Phase 3	Environment Setup	2 Days
Phase 4	Test Execution	10 Days

Phase	Activity	Duration
Phase 5	Regression Testing	5 Days
Phase 6	Test Closure	2 Days

10. Roles and Responsibilities

Role	Responsibility
Test Manager	Overall test strategy, planning, reporting
QA Lead	Test coordination, review, risk management
Test Engineer	Test case design, execution, defect logging
Automation Engineer	Automation scripting and maintenance
Developer	Bug fixing, unit testing
Business Analyst	Requirement clarification
UAT Users	Business validation

11. Defect Management Process

- Defects will be logged in JIRA
- Defect lifecycle:
 - New → Assigned → In Progress → Fixed → Retest → Closed
- Severity Levels:
 - S1 – Critical
 - S2 – High
 - S3 – Medium
 - S4 – Low

12. Risk and Mitigation

Risk	Mitigation
Environment instability	Use dedicated QA environment
Requirement gaps	Frequent review sessions
Test data issues	Prepare synthetic/masked data
Tight deadlines	Prioritize critical test cases
Integration failures	Early integration testing

13. Test Data Management

- Real data (controlled use)
- Masked data for sensitive info
- Synthetic data for edge cases
- Periodic refresh of test data

14. Configuration Management

- Version control for test scripts
- Build version tracking
- Environment configuration consistency
- CI/CD pipeline integration

15. Reporting and Metrics

Reports

- Daily Status Report
- Test Execution Report
- Defect Report
- Test Summary Report

Metrics

- Test Case Execution %
- Pass/Fail Rate
- Defect Density
- Defect Leakage

- Automation Coverage %

16. Approval

Role	Name	Signature
Test Manager		
QA Lead		
Project Manager		
Business Stakeholder		

Done – Enterprise-Level Test Plan Ready

This document is:

- Interview-ready 
 - Project-ready 
 - Aligned with real enterprise QA standards 
-

Next Step (Highly Recommended)

If you want to go even stronger for interviews or real work, I can generate:

-  RTM (Requirement Traceability Matrix)
-  Detailed Test Cases (Excel format)
-  Automation Framework (Selenium + Cucumber + TestNG)
-  CI/CD Jenkins Pipeline Sample

Just tell me 